New Horizons
Centro de Capacitación

Práctica: Modales de Aprobación/Rechazo/Pago y Componente Timeline de Auditoría

Paso a paso — Modales MatDialog con comentario obligatorio y componente visual Timeline que consume GET /api/solicitudes/{id}/timeline

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 12 — Pantallas y Modales del Flujo de Aprobación End-to-End
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Intermedio
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Construir diálogos modales con MatDialog para captura de comentarios de aprobación y motivos de rechazo.
- Implementar modal de Tesorería para registrar método de pago y número de operación bancaria.
- Crear el componente TimelineComponent que renderiza la cronología visual de transiciones.
- Consumir el endpoint GET /api/solicitudes/{id}/timeline de C# mapeando fecha, responsable y estado.
- Completar el flujo integral end-to-end desde la creación en Borrador hasta el Pago en Tesorería.

2. Desarrollo paso a paso

Paso 1 — Crear el Diálogo de Aprobación / Rechazo

Creamos DictamenDialogComponent en src/app/components/dictamen-dialog/ separando sus archivos .ts, .html y .css:

src/app/components/dictamen-dialog/dictamen-dialog.component.ts

```
import { Component, Inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { MatDialogModule, MAT_DIALOG_DATA, MatDialogRef } from '@angular/material/dialog';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatButtonModule } from '@angular/material/button';

export interface DictamenDialogData {
  tipo: 'aprobar' | 'rechazar';
  codigoSolicitud: string;
}

@Component({
  selector: 'app-dictamen-dialog',
  standalone: true,
  imports: [CommonModule, FormsModule, MatDialogModule, MatFormFieldModule, MatInputModule,
MatButtonModule],
  templateUrl: './dictamen-dialog.component.html',
  styleUrl: './dictamen-dialog.component.css'
})
export class DictamenDialogComponent {
  comentario: string = '';

  constructor(
    public dialogRef: MatDialogRef<DictamenDialogComponent>,
    @Inject(MAT_DIALOG_DATA) public data: DictamenDialogData
  ) {}
}
```

src/app/components/dictamen-dialog/dictamen-dialog.component.html

```
<h2 mat-dialog-title [class.text-danger]="data.tipo === 'rechazar'">
  {{ data.tipo === 'aprobar' ? 'Aprobar Solicitud' : 'Rechazar Solicitud' }}
{{ data.codigoSolicitud }}
</h2>
<mat-dialog-content>
  <p>
    {{ data.tipo === 'aprobar'
       ? 'Ingrese una observación o comentario conforme antes de autorizar el pago:'
       : 'Debe especificar el motivo o justificación técnica por la cual se rechaza:' }}
  </p>
  <mat-form-field appearance="outline" class="full-width">
    <mat-label>{{ data.tipo === 'aprobar' ? 'Comentario (opcional)' : 'Motivo de rechazo
(obligatorio)' }}</mat-label>
    <textarea matInput [(ngModel)]="comentario" rows="3" placeholder="Escriba
aquí..."></textarea>
  </mat-form-field>
</mat-dialog-content>
<mat-dialog-actions align="end">
  <button mat-button (click)="dialogRef.close(null)">Cancelar</button>
  <button mat-raised-button
          [color]="data.tipo === 'aprobar' ? 'primary' : 'warn'"
          [disabled]="data.tipo === 'rechazar' && !comentario.trim()"
          (click)="dialogRef.close(comentario)">
    {{ data.tipo === 'aprobar' ? 'Confirmar Aprobación' : 'Rechazar Definitivamente' }}
  </button>
</mat-dialog-actions>
```

src/app/components/dictamen-dialog/dictamen-dialog.component.css

```
.full-width { width: 100%; margin-top: 8px; }
.text-danger { color: #dc2626; }
```

Paso 2 — Crear el Diálogo de Pago de Tesorería

Creamos PagoDialogComponent en src/app/components/pago-dialog/ separando la lógica, plantilla HTML y estilos CSS:

src/app/components/pago-dialog/pago-dialog.component.ts

```
import { Component, Inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { MatDialogModule, MAT_DIALOG_DATA, MatDialogRef } from '@angular/material/dialog';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatSelectModule } from '@angular/material/select';
import { MatButtonModule } from '@angular/material/button';

@Component({
  selector: 'app-pago-dialog',
  standalone: true,
  imports: [CommonModule, FormsModule, MatDialogModule, MatFormFieldModule, MatInputModule,
MatSelectModule, MatButtonModule],
  templateUrl: './pago-dialog.component.html',
  styleUrl: './pago-dialog.component.css'
})
export class PagoDialogComponent {
  medioPago = 'Transferencia Bancaria';
  numeroOperacion = '';

  constructor(
    public dialogRef: MatDialogRef<PagoDialogComponent>,
    @Inject(MAT_DIALOG_DATA) public data: { codigo: string; monto: number }
  ) {}

  confirmar(): void {
    this.dialogRef.close({ medioPago: this.medioPago, numeroOperacion: this.numeroOperacion });
  }
}
```

src/app/components/pago-dialog/pago-dialog.component.html

```
<h2 mat-dialog-title>Registrar Pago en Tesorería</h2>
<mat-dialog-content>
  <p>Completar los datos del desembolso para la solicitud <strong>{{ data.codigo }}</strong>:</p>

  <mat-form-field appearance="outline" class="full-width">
    <mat-label>Medio de Pago</mat-label>
    <mat-select [(ngModel)]="medioPago">
      <mat-option value="Transferencia Bancaria">Transferencia Bancaria BCP/BBVA</mat-option>
      <mat-option value="Cheque de Gerencia">Cheque de Gerencia</mat-option>
      <mat-option value="Caja Chica">Caja Chica en Efectivo</mat-option>
    </mat-select>
  </mat-form-field>

  <mat-form-field appearance="outline" class="full-width">
    <mat-label>Número de Operación / Referencia</mat-label>
    <input matInput [(ngModel)]="numeroOperacion" placeholder="Ej: TR-2026-998812">
  </mat-form-field>
</mat-dialog-content>
<mat-dialog-actions align="end">
  <button mat-button (click)="dialogRef.close(null)">Cancelar</button>
  <button mat-raised-button color="primary" [disabled]="!numeroOperacion.trim()"
(click)="confirmar()">
    Confirmar y Cerrar Pago
  </button>
</mat-dialog-actions>
```

src/app/components/pago-dialog/pago-dialog.component.css

```
.full-width { width: 100%; margin-bottom: 12px; }
```

Paso 3 — Construir el Componente Visual Timeline

Implementamos TimelineComponent en src/app/components/timeline/ con sus tres archivos separados (.ts, .html y .css):

src/app/components/timeline/timeline.component.ts

```
import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { MatIconModule } from '@angular/material/icon';
import { TimelineItem } from '../../models/solicitud.model';

@Component({
  selector: 'app-timeline',
  standalone: true,
  imports: [CommonModule, MatIconModule],
  templateUrl: './timeline.component.html',
  styleUrl: './timeline.component.css'
})
export class TimelineComponent {
  @Input() items: TimelineItem[] = [];

  obtenerIcono(estado: string): string {
    switch (estado.toLowerCase()) {
      case 'borrador': return 'edit_note';
      case 'enviada': return 'send';
      case 'enrevision': return 'pending';
      case 'aprobada': return 'check_circle';
      case 'rechazada': return 'cancel';
      case 'pagada': return 'payments';
      default: return 'history';
    }
  }
}
```

src/app/components/timeline/timeline.component.html

```
<div class="timeline-container">
  <h3>Historial de Trazabilidad y Aprobaciones</h3>
  <div class="timeline-list">
    <div *ngFor="let item of items; let last = last" class="timeline-item">
      <div class="timeline-badge" [ngClass]="item.estadoNombre.toLowerCase()">
        <mat-icon>{{ obtenerIcono(item.estadoNombre) }}</mat-icon>
      </div>
      <div class="timeline-content">
        <div class="timeline-header">
          <span class="estado-tag">{{ item.estadoNombre }}</span>
          <span class="fecha">{{ item.fechaAccion | date:'dd/MM/yyyy HH:mm' }}</span>
        </div>
        <p class="responsable">Responsable: <strong>{{ item.responsableNombre }}</strong></p>
        <p *ngIf="item.comentario" class="comentario">
          <em>"{{ item.comentario }}"</em>
        </p>
      </div>
      <div *ngIf="!last" class="timeline-line"></div>
    </div>

    <div *ngIf="items.length === 0" class="empty-timeline">
      <p>No se registran transiciones previas para esta solicitud.</p>
    </div>
  </div>
</div>
```

src/app/components/timeline/timeline.component.css

```
.timeline-container { background: white; padding: 24px; border-radius: 12px; border: 1px solid
#e2e8f0; }
.timeline-list { position: relative; margin-top: 16px; padding-left: 10px; }
.timeline-item { position: relative; padding-bottom: 24px; padding-left: 40px; }
.timeline-badge {
  position: absolute; left: 0; top: 0; width: 32px; height: 32px; border-radius: 50%;
  display: flex; align-items: center; justify-content: center; color: white; background: #64748b;
}
.timeline-badge.enviada { background: #0284c7; }
.timeline-badge.enrevision { background: #f59e0b; }
.timeline-badge.aprobada { background: #10b981; }
.timeline-badge.rechazada { background: #ef4444; }
.timeline-badge.pagada { background: #6366f1; }
.timeline-line { position: absolute; left: 15px; top: 32px; bottom: 0; width: 2px; background:
#e2e8f0; }
.timeline-header { display: flex; justify-content: space-between; align-items: center; margin-
bottom: 4px; }
.estado-tag { font-weight: 600; font-size: 0.95rem; }
.fecha { font-size: 0.8rem; color: #64748b; }
.responsable { margin: 2px 0; font-size: 0.9rem; color: #334155; }
.comentario { margin: 6px 0 0; font-size: 0.85rem; color: #475569; background: #f8fafc; padding:
8px 12px; border-radius: 6px; }
.empty-timeline { text-align: center; color: #94a3b8; padding: 16px; }
```

Paso 4 — Integrar Acciones por Rol, Modales y Timeline en SolicitudDetailComponent

Actualizamos src/app/pages/solicitud-detail/ separando sus archivos .ts, .html y .css para conectar todo el flujo End-to-End:

`src/app/pages/solicitud-detail/solicitud-detail.component.ts`

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ActivatedRoute, RouterLink } from '@angular/router';
import { MatCardModule } from '@angular/material/card';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
import { MatTableModule } from '@angular/material/table';
import { MatDialog, MatDialogModule } from '@angular/material/dialog';
import { MatSnackBar } from '@angular/material/snack-bar';
import { SolicitudHttpService } from '../../services/solicitud-http.service';
import { RolService, RolUsuario } from '../../services/rol.service';
import { Solicitud, TimelineItem, EstadoSolicitud } from '../../models/solicitud.model';
import { DictamenDialogComponent } from '../../components/dictamen-dialog/dictamen-
dialog.component';
import { PagoDialogComponent } from '../../components/pago-dialog/pago-dialog.component';
import { TimelineComponent } from '../../components/timeline/timeline.component';

@Component({
  selector: 'app-solicitud-detail',
  standalone: true,
  imports: [
    CommonModule, RouterLink, MatCardModule, MatButtonModule, MatIconModule, MatTableModule,
    MatDialogModule, TimelineComponent
  ],
  templateUrl: './solicitud-detail.component.html',
  styleUrl: './solicitud-detail.component.css'
})
export class SolicitudDetailComponent implements OnInit {
  private route = inject(ActivatedRoute);
  private httpService = inject(SolicitudHttpService);
  private dialog = inject(MatDialog);
  private snackBar = inject(MatSnackBar);
  rolService = inject(RolService);
  RolUsuario = RolUsuario;

  solicitud?: Solicitud;
  eventosTimeline: TimelineItem[] = [];

  ngOnInit(): void {
    const id = Number(this.route.snapshot.paramMap.get('id'));
    this.cargarSolicitud(id);
    this.cargarTimeline(id);
  }

  cargarSolicitud(id: number): void {
    this.httpService.obtenerPorId(id).subscribe({
      next: (data) => { this.solicitud = data; }
    });
  }

  cargarTimeline(id?: number): void {
    const solId = id || this.solicitud?.id;
    if (!solId) return;
    this.httpService.obtenerTimeline(solId).subscribe({
      next: (eventos) => { this.eventosTimeline = eventos; }
    });
  }

  esRol(rol: RolUsuario): boolean {
    return this.rolService.usuarioActual().rol === rol;
  }

  noHayAccionesDisponibles(): boolean {
    if (!this.solicitud) return true;
    const s = this.solicitud;
    const r = this.rolService.usuarioActual().rol;
    if (s.estado === 1 && r === RolUsuario.Solicitante) return false;
    if (s.estado === 2 && r === RolUsuario.Revisor) return false;
    if (s.estado === 3 && r === RolUsuario.Aprobador) return false;
    if (s.estado === 4 && r === RolUsuario.Tesoreria) return false;
    return true;
  }

  enviarSolicitud(): void {
    if (!this.solicitud) return;
    this.httpService.enviar(this.solicitud.id).subscribe({
```

```
      next: (act) => {
        this.solicitud = act;
        this.cargarTimeline();
        this.snackBar.open('¡Solicitud enviada a revisión con éxito!', 'OK', { duration: 3000 });
      }
    });
  }

  iniciarRevision(): void {
    if (!this.solicitud) return;
    this.httpService.iniciarRevision(this.solicitud.id,
this.rolService.usuarioActual().id).subscribe({
      next: (act) => {
        this.solicitud = act;
        this.cargarTimeline();
        this.snackBar.open('Revisión técnica iniciada.', 'OK', { duration: 3000 });
      }
    });
  }

  abrirModalDictamen(tipo: 'aprobar' | 'rechazar'): void {
    if (!this.solicitud) return;
    const ref = this.dialog.open(DictamenDialogComponent, {
      width: '500px',
      data: { tipo, codigoSolicitud: this.solicitud.codigo }
    });

    ref.afterClosed().subscribe(comentario => {
      if (comentario !== null && comentario !== undefined) {
        const id = this.solicitud!.id;
        const usuarioId = this.rolService.usuarioActual().id;

        if (tipo === 'aprobar') {
          this.httpService.aprobar(id, comentario, usuarioId).subscribe({
            next: (act) => {
              this.solicitud = act;
              this.cargarTimeline();
              this.snackBar.open('¡Solicitud aprobada con éxito!', 'OK', { duration: 3000 });
            }
          });
        } else {
          this.httpService.rechazar(id, comentario, usuarioId).subscribe({
            next: (act) => {
              this.solicitud = act;
              this.cargarTimeline();
              this.snackBar.open('Solicitud rechazada.', 'OK', { duration: 3000 });
            }
          });
        }
      }
    });
  }

  abrirModalPago(): void {
    if (!this.solicitud) return;
    const ref = this.dialog.open(PagoDialogComponent, {
      width: '500px',
      data: { codigo: this.solicitud.codigo, monto: this.solicitud.montoTotal }
    });

    ref.afterClosed().subscribe(datosPago => {
      if (datosPago) {
        const id = this.solicitud!.id;
        const usuarioId = this.rolService.usuarioActual().id;
        this.httpService.pagar(id, datosPago.medioPago, datosPago.numeroOperacion,
usuarioId).subscribe({
          next: (act) => {
            this.solicitud = act;
            this.cargarTimeline();
            this.snackBar.open('¡Desembolso y pago registrado exitosamente en Tesorería!', 'OK',
{ duration: 3500 });
          }
        });
      }
    });
  }
}
```

New Horizons
Centro de Capacitación

`src/app/pages/solicitud-detail/solicitud-detail.component.html`

```
<div class="detail-container" *ngIf="solicitud">
  <div class="detail-header">
    <a mat-button routerLink="/solicitudes">&larr; Volver a la Bandeja</a>
    <div class="title-row">
      <h2>Solicitud {{ solicitud.codigo }}</h2>
      <span class="badge" [ngClass]="'badge-' + solicitud.estadoNombre.toLowerCase()">
        {{ solicitud.estadoNombre }}
      </span>
    </div>
  </div>

  <!-- BARRA DE ACCIONES CONDICIONADAS SEGUN ROL Y ESTADO -->
  <mat-card class="actions-card mat-elevation-z1">
    <div class="actions-wrapper">
      <span class="actions-info">
        Acciones disponibles para: <strong>{{ rolService.usuarioActual().nombre }}</strong>
      </span>

      <div class="buttons-group">
        <!-- 1. Enviar a Revisión (Solicitante en Borrador) -->
        <button mat-raised-button color="primary"
                *ngIf="solicitud.estado === 1 && esRol(RolUsuario.Solicitante)"
                (click)="enviarSolicitud()">
          <mat-icon>send</mat-icon> Enviar a Revisión
        </button>

        <!-- 2. Iniciar Revisión (Revisor en Enviada) -->
        <button mat-raised-button color="accent"
                *ngIf="solicitud.estado === 2 && esRol(RolUsuario.Revisor)"
                (click)="iniciarRevision()">
          <mat-icon>pending_actions</mat-icon> Iniciar Revisión
        </button>

        <!-- 3. Aprobar y Rechazar (Aprobador en EnRevision) -->
        <button mat-raised-button color="primary"
                *ngIf="solicitud.estado === 3 && esRol(RolUsuario.Aprobador)"
                (click)="abrirModalDictamen('aprobar')">
          <mat-icon>check_circle</mat-icon> Aprobar Solicitud
        </button>
        <button mat-stroked-button color="warn"
                *ngIf="solicitud.estado === 3 && esRol(RolUsuario.Aprobador)"
                (click)="abrirModalDictamen('rechazar')">
          <mat-icon>cancel</mat-icon> Rechazar
        </button>

        <!-- 4. Pagar (Tesorería en Aprobada) -->
        <button mat-raised-button color="primary" class="btn-pago"
                *ngIf="solicitud.estado === 4 && esRol(RolUsuario.Tesoreria)"
                (click)="abrirModalPago()">
          <mat-icon>payments</mat-icon> Procesar Pago
        </button>

        <span *ngIf="noHayAccionesDisponibles()" class="sin-acciones">
          No hay acciones pendientes para su rol en el estado actual.
        </span>
      </div>
    </div>
  </mat-card>

  <!-- DATOS GENERALES -->
  <mat-card class="detail-card mat-elevation-z2">
    <div class="info-grid">
      <div><span class="label">Motivo o Justificación:</span> <strong>{{ solicitud.motivo
}}</strong></div>
      <div><span class="label">Solicitante:</span> {{ solicitud.solicitanteNombre }}</div>
      <div><span class="label">Centro de Costo:</span> {{ solicitud.centroCostoNombre || 'Sin
asignar' }}</div>
      <div><span class="label">Fecha Creación:</span> {{ solicitud.fechaCreacion |
date:'dd/MM/yyyy HH:mm' }}</div>
      <div><span class="label">Monto Total:</span> <strong class="monto">{{ solicitud.montoTotal
| currency:'PEN':'symbol':'1.2-2' }}</strong></div>
    </div>

    <h3 class="section-title">Partidas de Gasto</h3>
    <table mat-table [dataSource]="solicitud.detalles" class="full-width-table">
      <ng-container matColumnDef="concepto">
```

```
        <th mat-header-cell *matHeaderCellDef> Concepto </th>
        <td mat-cell *matCellDef="let d"> {{ d.concepto }} </td>
      </ng-container>
      <ng-container matColumnDef="cantidad">
        <th mat-header-cell *matHeaderCellDef> Cantidad </th>
        <td mat-cell *matCellDef="let d"> {{ d.cantidad }} </td>
      </ng-container>
      <ng-container matColumnDef="precio">
        <th mat-header-cell *matHeaderCellDef> P. Unitario </th>
        <td mat-cell *matCellDef="let d"> {{ d.precioUnitario | currency:'PEN':'symbol':'1.2-
2' }} </td>
      </ng-container>
      <ng-container matColumnDef="importe">
        <th mat-header-cell *matHeaderCellDef> Subtotal </th>
        <td mat-cell *matCellDef="let d"> {{ d.importe | currency:'PEN':'symbol':'1.2-2' }} </td>
      </ng-container>

      <tr mat-header-row *matHeaderRowDef="['concepto', 'cantidad', 'precio', 'importe']"></tr>
      <tr mat-row *matRowDef="let row; columns: ['concepto', 'cantidad', 'precio',
'importe'];"></tr>
    </table>
  </mat-card>

  <!-- TIMELINE DE AUDITORIA Y TRAZABILIDAD -->
  <div class="timeline-wrapper">
    <app-timeline [items]="eventosTimeline"></app-timeline>
  </div>
</div>
```

src/app/pages/solicitud-detail/solicitud-detail.component.css

```
.detail-container { max-width: 950px; margin: 0 auto; padding: 12px; }
.title-row { display: flex; align-items: center; gap: 16px; margin-top: 8px; }
.actions-card { margin-bottom: 20px; padding: 16px 24px; border-radius: 10px; background:
#f0fdf4; border: 1px solid #bbf7d0; }
.actions-wrapper { display: flex; justify-content: space-between; align-items: center; flex-wrap:
wrap; gap: 12px; }
.actions-info { color: #166534; font-size: 0.95rem; }
.buttons-group { display: flex; gap: 12px; align-items: center; }
.btn-pago { background-color: #4f46e5 !important; color: white !important; }
.sin-acciones { font-size: 0.85rem; color: #64748b; font-style: italic; }
.detail-card { padding: 24px; border-radius: 12px; margin-bottom: 24px; }
.info-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-bottom: 24px; }
.label { color: #64748b; font-size: 0.9rem; display: block; }
.monto { font-size: 1.2rem; color: #024b98; }
.section-title { border-top: 1px solid #e2e8f0; padding-top: 16px; margin-top: 16px; }
.full-width-table { width: 100%; }
.timeline-wrapper { margin-top: 24px; }
.badge { padding: 4px 12px; border-radius: 16px; color: white; font-weight: 600; font-size:
0.85rem; }
.badge-borrador { background: #94a3b8; }
.badge-enviada { background: #0284c7; }
.badge-enrevision { background: #f59e0b; }
.badge-aprobada { background: #10b981; }
.badge-rechazada { background: #ef4444; }
.badge-pagada { background: #6366f1; }
```

3. Retos adicionales

1. Integra un validador en el modal de dictamen que prohíba comentarios genéricos como 'ok' o 'bien' para rechazos.
2. Añade un botón de impresión o generación de PDF del comprobante de solicitud pagada con el timeline incluido.

3. Permite adjuntar archivos de cotización o comprobantes de pago como simulación multipart en el frontend.
4. Verifica el flujo completo creando una solicitud desde cero hasta el estado Pagada con trazabilidad completa.

4. Entregables

- Modales de MatDialog funcionales para Aprobación, Rechazo con motivo obligatorio y Pago en Tesorería.
- Componente Timeline interactivo que consume en tiempo real GET /api/solicitudes/{id}/timeline de C#.
- Flujo de aprobación end-to-end completado exitosamente: Borrador -> Enviada -> EnRevision -> Aprobada/Rechazada -> Pagada.
- Persistencia e historial validado en la base de datos de ASP.NET Core.